

Hackathon FIAP X — Fase 5 — Entrega no portal do aluno (PDF)

Grupo: Oficina Turbo (106)

Aluno: Felipe Ricarte Magalhães

1. Dois repositórios de microsserviços (Fase 5 — FIAP X)

#	Repositório	Responsabilidade	Porta
1	https://github.com/ricartefelipe/fiapx-api-service	API REST — autenticação, upload, listagem de status, download do ZIP	8080
2	https://github.com/ricartefelipe/fiapx-processor-service	Worker assíncrono — extração de frames com FFmpeg e geração do ZIP	8081

Branches: `main` (estável, CI verde), `develop` (integração).

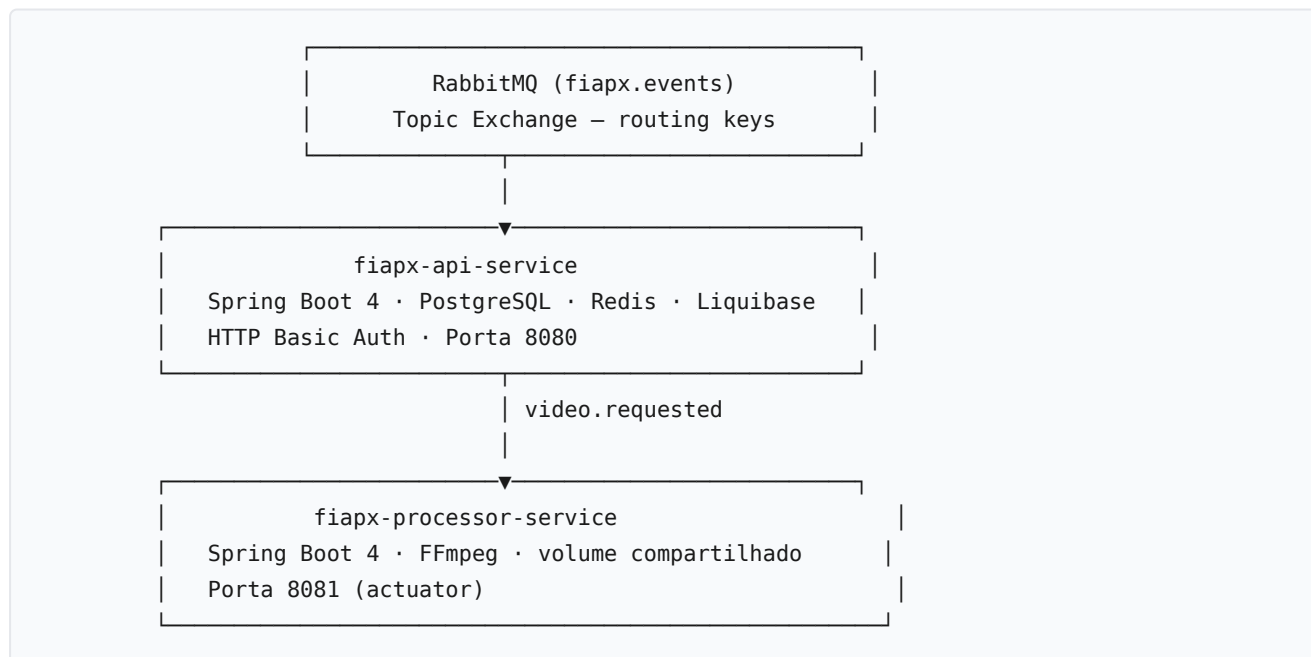
Acesso ao avaliador SOAT: usuário `soat-architecture` com leitura nos dois repositórios.

Documentação técnica no repositório central:

<https://github.com/ricartefelipe/fiapx-api-service/tree/main/docs/fase5>

2. Arquitetura — Microsserviços e fluxo de processamento

2.1 Visão geral



Infraestrutura compartilhada (Docker Compose): PostgreSQL, RabbitMQ, Redis, Prometheus e Grafana sobem junto com os dois microserviços.

2.2 Fluxo upload → fila → FFmpeg → ZIP → download

Etapa	Componente	Ação
1	Cliente	POST /api/videos (multipart) com HTTP Basic Auth
2	API Service	Persiste VideoJob (status QUEUED), salva arquivo e publica video.requested
3	RabbitMQ	Entrega mensagem na fila video.processing
4	Processor Service	Publica video.processing , extrai frames com FFmpeg, gera ZIP no volume compartilhado
5	Processor Service	Publica video.completed ou video.failed
6	API Service	Atualiza status do job; em erro, notifica via log e e-mail (MailHog)
7	Cliente	GET /api/videos e GET /api/videos/{id} ; download do ZIP quando COMPLETED

2.3 Eventos RabbitMQ

Exchange	Routing Key	Produtor	Consumidor
fiapx.events	video.requested	API	Processor
fiapx.events	video.processing	Processor	API
fiapx.events	video.completed	Processor	API
fiapx.events	video.failed	Processor	API

3. Stack técnica

Componente	Tecnologia
Linguagem	Java 21
Framework	Spring Boot 4.0.6
API REST	Spring Web + Spring Security (HTTP Basic Auth)
Mensageria	RabbitMQ + Spring AMQP
Banco relacional	PostgreSQL + Liquibase

Cache	Redis (listagem de jobs por usuário, TTL 5 min)
Processamento de vídeo	FFmpeg (via ProcessBuilder no processor)
Documentação API	SpringDoc OpenAPI / Swagger UI
Testes	JUnit 5 + Mockito + Testcontainers (PostgreSQL, RabbitMQ, GreenMail)
Cobertura	JaCoCo (gate $\geq 70\%$ no CI)
CI/CD	GitHub Actions
Containers	Docker + Docker Compose
Observabilidade	Micrometer + Prometheus + Grafana

4. Requisitos do hackathon — checklist

Matriz detalhada (requisito $\times$ status $\times$ evidência $\times$ gap): [docs/fase5/matriz-conformidade.md](https://docs.fase5.com.br/matriz-conformidade.md)

Requisito	Status	Evidência / gap
Processar mais de um vídeo simultaneamente	Conforme	Fila + worker assíncrono; <code>scripts/verify-conformidade.sh</code> (2 uploads paralelos)
Não perder requisições em picos de carga	Conforme	Filas duráveis; API retorna 202 Accepted
Proteção por usuário e senha	Conforme	HTTP Basic Auth (<code>fiapx</code> / <code>fiapx123</code>); testes de 401 e isolamento por usuário
Listagem de status dos vídeos por usuário	Conforme	<code>GET /api/videos</code> e <code>GET /api/videos/{id}</code>
Notificação em caso de erro	Conforme	<code>CompositeNotificationService</code> + e-mail (MailHog); teste integração GreenMail
Persistência de dados	Conforme	PostgreSQL (<code>users</code> , <code>video_jobs</code>)
Arquitetura escalável	Conforme	API e processor desacoplados via fila
Versionamento no GitHub	Conforme	Dois repositórios com GitFlow
Testes automatizados	Conforme	<code>./mvnw -Pci clean verify</code> ; 39 testes; Testcontainers; JaCoCo $\geq 70\%$ (gate CI)
CI/CD	Parcial	CI (build, testes, imagem GHCR); sem pipeline de deploy
Docker Compose funcional	Conforme	<code>docker compose up -d --build</code>

Teste E2E (fluxo feliz)	Conforme	<code>./scripts/e2e-test.sh</code>
Verificação ampliada (paralelo, falha, mail, métricas)	Conforme	<code>./scripts/verify-conformidade.sh</code>
Vídeo de apresentação (≤ 10 min)	Conforme	<code>fiapx-fase5-demo.mp4</code> incluído no <code>fase5.zip</code> (~4m46s, sem áudio)
Evolução do projeto base FIAP	Parcial	Reimplementação equivalente, não fork do repo base

5. Cobertura de testes e CI

5.1 JaCoCo (local, perfil `ci`)

Serviço	Cobertura de instruções	Testes
fiapx-api-service	~90%	Unitários + integração Testcontainers (PostgreSQL, RabbitMQ, GreenMail) — 29 testes
fiapx-processor-service	~97%	Unitários + integração RabbitMQ + FFmpeg simulado — 10 testes

Cobertura medida com `./mvnw -Pci clean verify`. CI falha se JaCoCo cair abaixo de **70%**. Integração cobre listeners, fluxo de mensagens, segurança HTTP Basic e notificação por e-mail.

5.2 CI/CD — GitHub Actions

Repositório	Workflow	Status em <code>main</code>
fiapx-api-service	<code>ci.yml</code> + <code>enforce-gitflow.yml</code>	✓ Verde
fiapx-processor-service	<code>ci.yml</code> + <code>enforce-gitflow.yml</code>	✓ Verde

Evidências:

- <https://github.com/ricartefelipe/fiapx-api-service/actions>
- <https://github.com/ricartefelipe/fiapx-processor-service/actions>

Imagens Docker publicadas em: `ghcr.io/ricartefelipe/fiapx-api-service` e `ghcr.io/ricartefelipe/fiapx-processor-service`

5.3 SonarCloud

Item	Status
------	--------

Propriedades no pom.xml	✓ Configurado (ricartefelipe_fiapx-api-service , ricartefelipe_fiapx-processor-service)
Análise automática no CI	Conforme — SonarCloud Code Analysis nos PRs
Dashboards	https://sonarcloud.io/project/overview?id=ricartefelipe_fiapx-api-service

6. Como executar

6.1 Pré-requisitos

- Docker e Docker Compose
- Repositórios `fiapx-api-service` e `fiapx-processor-service` no mesmo diretório pai
- Opcional: FFmpeg no host (para o script E2E gerar vídeo de teste)

6.2 Stack completa (recomendado)

```
cd fiapx-api-service
docker compose up -d --build
```

Serviço	URL / Porta
API	http://localhost:8080/api
Swagger UI	http://localhost:8080/api/swagger-ui.html
Processor (health)	http://localhost:8081/actuator/health
RabbitMQ Management	http://localhost:15672 (guest/guest)
Prometheus	http://localhost:9090
Grafana	http://localhost:3000 (admin/admin) — dashboard FIAP X — Visão geral
MailHog (e-mail de falha)	http://localhost:8025

Credenciais da API: `fiapx` / `fiapx123`

6.3 Teste E2E automatizado (fluxo feliz)

```
cd fiapx-api-service
chmod +x scripts/e2e-test.sh
./scripts/e2e-test.sh
```

O script aguarda a API, envia upload, poll até `COMPLETED` e valida o download do ZIP.

6.4 Verificação de conformidade ampliada

```
cd fiapx-api-service
chmod +x scripts/verify-conformidade.sh
./scripts/verify-conformidade.sh
```

Valida: health da API (8080) e processor (8081), dois uploads paralelos, falha com arquivo inválido + e-mail no MailHog, targets Prometheus UP.

6.5 Testes unitários

```
./mvnw -Pci clean verify # em cada repositório
```

7. GitFlow aplicado

Fluxo documentado em docs/GITFLOW.md (ambos os repositórios):

Branch	Uso
feature/*	Novas funcionalidades → PR para develop
fix/*	Correções → PR para develop
release/*	Preparar versão → PR develop → main
hotfix/*	Correção urgente → PR para main + sync develop

O workflow enforce-gitflow.yml **bloqueia** PR de feature/* direto para main.

Última release: testes de integração e gate JaCoCo — PR #21 (develop → main), CI verde em 29/08/2026.

8. Observabilidade

Ferramenta	Acesso
Prometheus	http://localhost:9090
Grafana	http://localhost:3000
Métricas	/actuator/prometheus em API (8080) e Processor (8081)

9. Vídeo demonstrativo (≤ 10 minutos, sem áudio)

Arquivo local: docs/delivery/fiapx-fase5-demo.mp4 (~4 min 46 s, mudo)

Como regenerar:

```
docker compose up -d --build
python3 scripts/capture-demo-frames.py
SECONDS_PER_FRAME=22 ./scripts/generate-demo-video.sh
```

Conteúdo: arquitetura, capturas de Swagger/RabbitMQ/MailHog/Grafana/Prometheus, upload, processamento, erro + e-mail, testes e GitHub.

Portal: enviar o MP4 ou publicar no YouTube/Vimeo e colar o link no campo do aluno.

10. Pacote zipado (portal)

Arquivo **fase5.zip** (upload no portal) contém a pasta **fase5/** com:

Arquivo	Conteúdo
entrega-portal-fase5.pdf	Documentação + arquitetura + checklist
fiapx-fase5-demo.mp4	Vídeo demo (~4m46s, sem áudio)
fase5-microservicos.zip	Código dos dois repositórios + cópia do PDF

Regenerar com: ./scripts/build-fase5-delivery.sh (valida PDF, vídeo e código antes de concluir).

Entrega Fase 5 — Hackathon FIAP X — plataforma de processamento de vídeos em microserviços com RabbitMQ, PostgreSQL e Docker Compose.